

4.2 String Class

Sue has just finished the Mad Lib® assignment and found working with strings to be tedious and problematic. There were so many ways to forget the NULL character! She decides to spend a few minutes and create her own string type so she never has to make that mistake again. As she sits in the lab drafting a solution, Sam walks in and peers over her shoulder. “You know,” he says “there is already a tool that does all those things...”

Objectives

By the end of this class, you will be able to:

- Use the String Class to solve a host of text manipulation problems.
- Understand which string operations are expensive and which are not.

Prerequisites

Before reading this section, please make sure you are able to:

- Understand the role the NULL character plays in string definitions (Chapter 3.2).
- Write a loop to traverse a string (Chapter 3.2).

Overview

One of the principles of Object Oriented programming (the topic of CS 165) is encapsulation, the process of hiding unimportant implementation details from the user of a tool so he can focus on how the tool can be used to solve his problem. To date, we have used two tools exemplifying this property: input file streams (`cin` and `fin`) and output file streams (`cout` and `fout`). We learned how to use these tools to solve programming problems, but never got into the details of how they work. Another similarly powerful tool is the String Class.

The String Class is a collection of tools defined in a library allowing us to easily manipulate text. With the String Class, the programmer does not need to worry about buffer sizes (we were using 256 up to this point), NULL characters, or using loops to copy text. In fact, most of the most common operations work as though they are operating on a simple data type (such as an integer), allowing the programmer to forget he is even working with arrays. Consider the following simple example:

```
#include <string>                                // use the string library

/*****
 * DEMO: simple string-class demo
 *****/
void demo()
{
    string lastName;                             // the data-type is "string," no []s
    cout << "What is your last name? ";          //      as were needed with c-strings
    cin  >> lastName;                             // cin works the way you expect

    string fullName = "Mr. " + lastName;          // the + operator appends

    cout << "Hello " << fullName << endl;        // cout works the way you expect
}
```

Syntax

The syntax of the String Class is designed to be as streamlined and intuitive as possible. There are several components: the string library, declaring a string, interfacing with streams, and performing common text manipulation operations.

String library

Unlike c-strings (otherwise known as “the array type for strings”), the String Class is not built into the C++ language. These tools are provided in the string library as part of the standard namespace. Therefore, it is necessary to include the following at the top of your programs:

```
#include <string>
```

If this line is omitted, the following compiler message will appear:

```
example.cpp: In function “int main()”:  
example.cpp:4: error: “string” was not declared in this scope
```

Note that our compiler actually includes the string library as part of iostream. This, technically, is not part of the iostream library design. We should never rely on this quirk of our current compiler library and always include the string library when using the String Class.

Declaring a string

With a c-string and all other built-in data-types in C++, variables are not initialized when they are declared. This is not true with the String Class. The act of declaring a string variable also initializes it to the empty string.

```
{  
    string text1;           // initialized to the empty string  
    cout << text1;         // displays the empty string: nothing  
  
    char text2[256];        // not initialized  
    cout << text2;         // random data will be displayed  
}
```

Observe how we do not specify the size of a string when we declare it. This is because the authors of the String Class did not want you to have to worry about such trivial details. Initially, the size is zero. However, as more data is added to the string, it will grow. We can always ask our string variable for its current capacity:

```
{  
    string textEmpty;  
    string textFull = "Introduction to Software Development";  
  
    cout << textEmpty.capacity() << endl;           // 0: the buffer is currently empty  
    cout << textFull.capacity() << endl;           // 64: the first power of 2 greater  
}
```

Sam's Corner

The String Class buffer is dynamically allocated. This allows the buffer to grow as the need arises. It also means that we need to use the capacity() function to find the size rather than sizeof(). When the string variable falls out of scope, its storage capacity is automatically freed unless it has been allocated with new.



Stream interfaces

We were able to use `cin` and `cout` with all the built-in data-types (`int`, `float`, `bool`, `char`, `double`, etc.), but not with arrays (but we are able to use them with individual elements of arrays). The one exception to this is c-strings; `cin` and `cout` treat pointers-to-characters as c-strings. When we include the string library, `cin` and `cout` are also able to work with string variables:

```
{
    string text;

    cin  >> text;           // works the same as a c-string
    cout << text << endl;   // both cin and cout work
}
```



Sue's Tips

When working with c-strings, we had to be careful to not put more data in the buffer than there was room. This was problematic with c-strings, unfortunately, because there was no way to tell `cin` how big the string is:

```
char text[10];
cin >> text;    // BUG! The user can enter more than 9 characters
```

With the String Class, the buffer size grows to accommodate the user input. This means that it is impossible for the user to specify more input than there is space in the buffer; the buffer will simply grow until it is big enough.

```
string text;
cin >> text;    // Safe! text can accommodate any amount
                // of user input
```

With c-strings, we can use `getline` to fetch an entire line of text. We can also use `getline` with the String Class, but the syntax is quite odd:

```
{
    // first the c-string syntax
    char text1[256];           // don't forget the buffer
    cin.getline(text1, 256);   // the buffer size is a required parameter

    // now the String Class
    string text2;              // no buffer needed here
    getline(cin, text2);       // note how cin is the parameter!
}
```

This syntax is, unfortunately, something we will just have to remember.

Sam's Corner



The reason for the String Class' strange `getline` syntax is a bit subtle. All the functionality associated with `cin` and `cout` are in the `iostream` library. If the `getline` method associated with `cin` took a string as a parameter, then the `iostream` library would need to know about the String Class. The `iostream` library must be completely ignorant of the String Class; otherwise everyone would be required to include the string library when they do any console I/O. This would make the coupling between the libraries tighter than necessary.

The string library extends the definition of `cin` and `cout`. We will learn more about how this is done in CS 165 where we learn to make our own custom data-types work with `cin` and `cout`.

Text manipulation

Great pains was taken to make text manipulation with `string` objects as easy and convenient as possible. We can append with the plus operator:

```
{
    string prefix = "Mr. ";
    string postfix = "Smith";

    string name = prefix + postfix;    // concatenation with the + operator. There is a
}                                     //   FOR loop hidden here. See Sue's tip below
```

We can copy strings with the equals operator:

```
{
    string text = "CS 124";
    string copy;

    copy = text;                      // copy with the = operator. There is a FOR loop
}                                     //   hidden here. See Sue's tip below.
```

We can compare with the double-equals operator: Note that `>`, `>=`, `<`, `<=` and `!=` work as you would expect.

```
{
    string text1;
    string text2;

    cin >> text1 >> text2;

    if (text1 == text2)                // same with the other comparison operators
        cout << "Same!\n";           //   such as == != > >= < <=
    else                               //   There is a FOR loop hidden here!
        cout << "Different!\n";       //   See Sue's tip below for a hint.
}
```

Finally, we can retrieve individual members of a string with the square-brackets operator:

```
{
    string text = "CS 124";

    for (int i = 0; i < 6; i++)
        cout << text[i] << endl;    // access data with the [] operator;
}                                     //   this is very fast! No FOR loops here
```

For more functionality associated with the `String` Class, please see:

http://en.cppreference.com/w/cpp/string/basic_string



Sue's Tips

Be careful of hidden performance costs when working with the `String` Class. Seemingly innocent operations (like `=` or `==`) must have a `FOR` loop in them. Ask yourself “if this were done with `c-strings`, would a `LOOP` be required?”

Please see the following example for a test of the performance of the `append` operator in the `String` Class at [4-2-stringPerformance.cpp](#) or:

```
/home/cs124/examples/4-2-stringPerformance.cpp
```

Example 4.2 – String Demo

Demo

This example will demonstrate how to declare a string, accept user input into a string, append text onto the end of a string, copy a string, and display the output on the screen.

Solution

All these common string operations will be demonstrated in a single function:

```
#include <iostream>
#include <string>          // don't forget this library
using namespace std;

/*****
 * MAIN: Simple demo of the string class.
 *****/
int main()
{
    string firstName;    // no []s required. The string takes care of the buffer
    string lastName;

    // cin and cout work as you would expect with the string class
    cout << "What is your name (first last): ";
    cin >> firstName >> lastName;

    // Append ", " after last name so "Young" becomes "Young, ".
    // To do this, we will use the += operator.
    lastName += ", ";

    // Create a new string containing the first and last name so
    // "Brigham" "Young"
    // becomes
    // "Young, Brigham".
    // To do this we will use the + operator to combine
    // two strings and the = operator to assign the results to a new string.
    string fullName = lastName + firstName;

    // display the results of our nifty creation
    cout << fullName << endl;

    return 0;
}
```

Challenge

As a challenge, change the above program to prompt the user for his middle name. Append this new string into fullName so we get the expected output.

See Also

The complete solution is available at [4-2-stringDemo.cpp](#) or:

/home/cs124/examples/4-2-stringDemo.cpp



Using string objects as file name variables

One might be tempted to use the string class for a file name. This makes the `getFilename()` function more intuitive and elegant:

```
/* *****  
 * GET FILENAME  
 * Prompt the user for a filename and return it  
 * ***** */  
string getFilename()                // no pass-by-pointer parameter!  
{  
    string fileName;                // local variable  
    cout << "Please provide a filename: ";  
    cin >> fileName;  
    return fileName;                // we can return a local variable  
}                                   // without fear it will be destroyed
```

From this point, how do we use the string `fileName`? Consider the following code:

```
{  
    string fileName = getFilename();    // this works as you might expect  
    ifstream fin(fileName);            // ERROR! Wrong parameter!  
}
```

The error is:

```
example.cpp:215: error: no matching function for call to "std::basic_ofstream<char,  
std::char_traits<char> >::open(std::string&)" /usr/lib/gcc/x86_64-redhat-  
linux/4.4.5/../../../../include/c++/4.4.5/fstream
```

This means that the file name parameter needs to be a c-string, not a string class. To do this, we need to convert our string variable in to a c-string variable. This is done with the `c_str()` function:

```
{  
    string fileName = getFilename();  
    ifstream fin(fileName.c_str());    // c_str() returns a pointer to a char  
}
```

Example 4.2 – String Properties

Demo	This example will demonstrate how to manipulate a string class object in a similar way to how we did this with c-strings. This will include using pointers as well as using the array notation to traverse a string.
Problem	Write a program to prompt the user for some text, display the number of characters, the number of spaces, and the contents of the string backwards. <pre>Please enter some text: <u>Software Development</u> Number of characters: 20 Number of spaces: 1 Text backwards: tnempoleveD erawtfoS</pre>
Solution	We can get the length of a string with: <pre>// find the number of characters cout << "\tNumber of characters: " << text.length() << endl;</pre> We can traverse the string using a pointer notation by getting a pointer to the start of the string with the <code>c_str()</code> method. This will return a constant pointer to a character. <pre>// find the number of spaces int numSpaces = 0; for (const char *p = text.c_str(); *p; p++) if (*p == ' ') numSpaces++;</pre> Finally, we can traverse the string using the array index notation: <pre>// display the string backwards. cout << "\tText backwards: "; for (int i = text.length() - 1; i >= 0; i--) cout << text[i]; cout << endl;</pre>
Challenge	As a challenge, try to change the case of all the characters in the string. This means converting uppercase characters to lowercase, and vice-versa. As you may recall, we did this earlier (please see p. 249). As another challenge, try to count the number of digits in the string. You may need to use <code>isdigit()</code> from the <code>cctype</code> library to accomplish this.
See Also	The complete solution is available at 4-2-stringProperties.cpp or: <pre>/home/cs124/examples/4-2-stringProperties.cpp</pre> 

Problem 1

Declare three string class variables. Prompt the user for the values and display them:

```
Please specify a name: Sam
Please specify another name: Sue
Please specify yet another name: Sid
The names are: "Sam" "Sue," and "Sid"
```

Answer:

Please see page 338 for a hint.

Problem 2

From the code written for Problem 1, create a new string called `allNames`. This string will be created in the following way: first name, ", ", second name, ", and ", and third name. Display the new string.

```
Please specify a name: Sam
Please specify another name: Sue
Please specify yet another name: Sid
The names are: "Sam, Sue, and Sid"
```

Answer:

Please see page 341 for a hint.

Problem 3

From the code written for Problem 2, append the strings in alphabetical order to `allName`. You will need to use the `>` operator to compare strings, which works much like it does with integers.

```
Please specify a name: Sam
Please specify another name: Sue
Please specify yet another name: Sid
The sorted names are: "Sam, Sid, and Sue"
```

Answer:

Please see page 340 for a hint.

Assignment 4.2

Consider the child's song "Dem Bones" found at (http://en.wikipedia.org/wiki/Dem_Bones or <http://www.youtube.com/>). Sue would like to write a program to display the first eight verses of the song. However, realizing that the song is highly repetitive, she would like to write a function to help her with the task.

Please write a function to generate the Dem Bones song. This function takes an array of strings as input and returns a single string that constitutes the entire song as output:

```
string generateSong(string list[], int num);
```

Consider the case where `num == 4` and `list` has the following items: `toe`, `foot`, `knee`, and `hip`. This will generate the following string:

```
toe bone connected to the foot bone  
foot bone connected to the knee bone  
knee bone connected to the hip bone
```

Directions

For this problem, the stub function `generateSong()` as well as `main()` is written for you. Your job is to implement the function `generateSong()`. The file is located at:

```
/home/cs124/assignments/assign42.cpp
```

Please start with the above file because **most of the program is written for you!**

Assignment

The test bed is available at:

```
testBed cs124/assign42 assignment42.cpp
```

Don't forget to submit your assignment with the name "Assignment 42" in the header.

Please see page 341 for a hint.